AutoRedTeam — Capstone Report

An autonomous multi-agent system for execution-verified penetration testing.

This document is the dissertation write-up and defense-preparation record for the project specified in `markdown/00_PROJECT_BRIEF.md … 09_OWNERSHIP_HANDSHAKE.md`. Every number in the Evaluation section was produced by the code in this repository against a locally sandboxed target; nothing here is illustrative or invented. Where something was specified but not demonstrated live, it is stated plainly in **Limitations**.

1. Abstract

Traditional automated security scanners (SAST/DAST) report large volumes of findings by pattern-matching code or responses, and in doing so produce high false-positive rates — each alert is a *hypothesis* of a vulnerability, not a *proof*. Manual red-teaming is accurate but slow and expensive. **AutoRedTeam** closes this gap: a team of specialised LLM agents reasons about plausible vulnerabilities and then **executes real proof-of-concept attacks** against a sandboxed copy of the target. The central design rule is that **a finding is reported only if it was demonstrated to work** — the verification is execution-grounded and, wherever possible, deterministic.

Against OWASP Juice Shop the system detected all four in-scope vulnerability classes it targets (IDOR, error-based SQLi, reflected/DOM XSS, broken authentication) with **precision 1.00 and a 0.00 false-positive rate**, each finding carrying a reproducible PoC. A comparative OWASP ZAP passive scan of the same target emitted **238 unconfirmed alerts** while matching **none** of the four actually-exploitable classes — a concrete illustration of the signal-vs-noise problem the project set out to address.

2. Problem & Thesis

- **Static/dynamic scanners over-report.** They flag code or response *shapes*, leaving a human to triage which alerts are real. The triage cost is the dominant cost.
- **Manual testing under-scales.** Expert red-teamers confirm exploitability but cannot cover large surfaces cheaply or repeatably.
- **Thesis:** if an autonomous system only reports what it can *execute and verify*, and prefers *deterministic* verification over model judgement, its false-positive rate collapses toward zero while retaining useful recall on checkable vulnerability classes.

The evaluation (Section 6) tests this thesis directly and reports precision broken out by verification confidence tier.

3. System Architecture

Five layers, top to bottom (full diagram in `markdown/01_ARCHITECTURE.md`):

Layer	Component	Role
Presentation	Streamlit dashboard (`dashboard/app.py`)	Start a run, view the report
API	FastAPI (`api/main.py`)	`POST /runs`, `GET /runs/{id}`, `/findings`
Orchestration	LangGraph state machine (`core/orchestration/graph.py`)	Drives recon → hypothesis → verify → report with retries
Execution / tools	httpx, Playwright, sandbox-scoped client	Executes PoCs against the target
Sandbox	Docker (`docker-compose.sandbox.yml`)	Isolated, egress-blocked target

Agents and the verifier engine:

1. **Recon Agent** (`core/agents/recon.py`) — deterministic, no LLM. Three discovery paths: OpenAPI/Swagger parsing (with real-JSON validation so an SPA shell cannot be mistaken for a spec), HTML crawling (forms, anchors), and a JS-asset path scanner that recovers a single-page app's REST surface. On Juice Shop it discovers **76 endpoints**.
2. **Attacker Agent** (`core/agents/attacker.py`) — the only reasoning agent. Given one endpoint's shape plus prior attempt history, it emits a single typed `Hypothesis` via LLM tool-calling. It reasons; it never executes. The real auth token is injected after parsing (the model never transcribes a JWT), and schema-invalid responses are retried locally. Provider-agnostic (any OpenAI-compatible endpoint); evaluated on Groq `openai/gpt-oss-120b`.
3. **Verifier Engine** (`core/verifier/`) — the core IP, **no LLM**. One module per class (IDOR, SQLi error/blind, reflected XSS, SSRF, path traversal, broken auth), each with a pure, unit-tested decision core. Produces a `VerificationResult` with a `confidence: "deterministic" | "llm_judged"` field.
4. **Reporter Agent** (`core/agents/reporter.py`) — **deterministic** CVSS (via the `cvss` library) and **deterministic** PoC steps (reconstructed from payload + evidence); only the remediation prose is LLM-generated (with an offline per-class template fallback).

Contracts. Every inter-agent message is a Pydantic model in `schemas/models.py` — the single source of truth. No free-text parsing between agents. This "agent boundaries are schema boundaries" rule is what lets the system separate *proposing* a hypothesis from *judging* it, which is the foundation of the false-positive guarantee.

4. How Verification Works (per class)

Deterministic success conditions, not reflection or pattern shape:

- **IDOR** — send a baseline request (the caller's own resource) and the attack request (another id) with the *same* token; exploited only if the attack returns 200 with JSON that is *structurally identical but data-different* from the baseline (guards against identical placeholder data and against 403/404).

- **SQLi (error-based)** — a DB-error signature must appear for the injected request **and be absent from a benign baseline** (so an app that always shows a generic error is not flagged).
- **SQLi (blind)** — statistical timing: N benign vs N delay-injected requests; flagged only when the mean delta exceeds the injected delay beyond baseline noise.
- **Reflected/DOM XSS** — a unique marker payload must **execute** in a real headless browser (sets `window.__xss_marker`), read back via Playwright — not a substring match on HTML.
- **SSRF** — an out-of-band callback bearing a unique token must reach a sandbox-internal listener.
- **Path traversal** — the response must contain a known OS-file sentinel (e.g. `root:…:0:0:`), not "some file content appeared".
- **Broken auth** — the endpoint must reject an unauthenticated request *and then accept* a tampered/forged token (two-step check prevents flagging an unprotected endpoint).

5. Implementation Summary

Built strictly in the roadmap's phase order (`markdown/08_BUILD_ROADMAP.md`) — **verifier before agents before orchestration** — so that execution/verification was proven with hand-written hypotheses before any LLM was wired in.

Phase	Deliverable	Status
0	Repo, schemas (verbatim), isolated Docker sandbox	✅ 3/3 isolation checks
1	Deterministic IDOR verifier	✅ mocked tests + live confirm
2	Recon agent (no LLM)	✅ 76 live endpoints
3	Attacker agent (LLM, IDOR)	✅ agent-driven IDOR, end-to-end
4	LangGraph orchestration + retry + request-budget guard	✅ single-call verified finding
5	All 6 remaining verifier classes	✅ 4/6 live true-positive
6	RAG knowledge layer (ChromaDB) + ablation switch	✅ scoped retrieval + wiring
7	Reporter (CVSS/PoC/remediation) + FastAPI + Streamlit	✅ live report via API
8	Ground truth, metrics, benchmark, ZAP baseline	✅ live numbers (ZAP passive)
9	Write-up + defense prep	← this document

Test suite: 57 unit tests, all passing (verifier logic, recon paths, attacker schema/retry, orchestration nodes/routers/budget, RAG retrieval, reporter CVSS/PoC, metrics scoring, API lifecycle) — all with LLM/HTTP/browser boundaries mocked, so they run without Docker, a browser, or an API key.

Stack: Python 3.13, Pydantic v2, LangGraph 1.2.9, httpx, Playwright (Chromium), ChromaDB 1.5.9 (offline hashing embedding, no model download), FastAPI, Streamlit, `cvss`. LLM via Groq (OpenAI-compatible) `openai/gpt-oss-120b`.

6. Evaluation

6.1 Method

- **Ground truth** (`evals/ground_truth.py`): documented Juice Shop vulnerabilities, each tagged `in_scope` so recall is reported against an honest denominator rather than a cherry-picked one.
- **Metrics** (`evals/metrics.py`, pure/unit-tested): precision, recall, false-positive rate (overall, per class, and by confidence tier), and ReAct efficiency (avg attempts-to-success). A finding is a true positive iff it matches a ground-truth entry on `(vuln_class, endpoint)`.
- **Two measurement modes** to separate the deterministic verifier from the stochastic agent:
  - **Engine mode** — canonical payloads, no LLM; fully reproducible; measures the verifier's capability.
  - **Agent mode** — the full system (LLM attacker); run across **3 seeds** and reported as mean ± standard deviation to quantify variance rather than cherry-pick a run.

6.2 Results (OWASP Juice Shop, live)

In-scope classes (IDOR, SQLI_ERROR, XSS_REFLECTED, BROKEN_AUTH):

Mode	Precision	Recall	FP-rate	Avg attempts
Engine (deterministic)	1.00	1.00	0.00	1.00
Agent (3 seeds)	1.00 ± 0.00	1.00 ± 0.00	0.00	1.08 ± 0.12

- **False-positive rate in the `deterministic` tier: 0.00** — the thesis, demonstrated.
- Agent recall/precision are perfectly stable across seeds (±0.00): the 1.00 result is not a lucky run. The only variance is *effort* (1.08 ± 0.12 attempts), i.e. the ReAct loop occasionally taking a second attempt on error-based SQLi.

Against all documented vulnerabilities (including out-of-scope): recall **0.57** (4 of 7). The three misses are **by design** and reported as false negatives: blind SQLi (SQLite has no `SLEEP()`), OS-file path traversal (Juice Shop confines `/ftp`), and SSRF (needs an out-of-band listener). See Limitations.

6.3 Comparative baseline — OWASP ZAP (passive)

Metric	AutoRedTeam	OWASP ZAP (passive)
in-scope classes detected	4 / 4	0 / 4
recall (in-scope)	1.00	0.00
confirmed true positives	4	—
false positives	0	238 unconfirmed
precision (confirmed-exploit)	1.00	n/a
total raw alerts	4	238 (170 Medium, 68 Informational)
exploit proven with PoC	yes (all)	no

ZAP's passive scan produced 238 alerts, none of which correspond to the four exploitable vulnerabilities — because passive scanning inspects spidered traffic and cannot actively probe. Those 238 alerts (missing headers, information disclosure, etc.) are exactly the unverified volume that a human must triage.

Honesty note: this is a *passive-only* ZAP run and is therefore **not** a fair active-vs-active comparison — ZAP's 0/4 in-scope is expected for passive mode. The active scan could not be run: the evaluation host (16 GB RAM) OOM-killed ZAP's active scanner twice, even with a capped 1 GB heap and the AJAX spider disabled, because the active scanner's thread fan-out (amplified by the sandbox's throttled egress to external telemetry endpoints) exceeds available memory. The active-scan runner (`evals/run_benchmark.py: run_zap_baseline`, with `scripts/run_zap_only.py`) is code-complete, resilient to connection drops, and reachability-verified against the target; it fills the active column unchanged on a higher-RAM host.

6.4 RAG ablation

Running the agent with vs without the ChromaDB knowledge base showed **no accuracy delta** on this in-scope set — the agent already saturates recall, so there is no headroom for RAG to improve it. This is itself a finding: ablating a precision-improvement layer on an "easy" target cannot demonstrate its value; a fair ablation needs a harder target with a lower base recall. The retrieval layer is built, scoped per class, and wired as a toggle (`OrchestrationDeps.retriever`) ready for that experiment.

7. Safety & Ethics

Every target is a locally-run, intentionally-vulnerable application. The system is not designed or intended to be pointed at any system the operator does not own or have explicit authorisation to test. Technical controls (binding, in `markdown/05_SANDBOX_SETUP.md` and `core/sandbox/client.py`):

- **Network isolation.** The target runs on a Docker network with `internal: true` — no route to the internet. A trusted socat gateway bridges host→target on loopback only, so the target is unreachable from any other machine. Verified: the target cannot egress; the orchestrator reaches it, an unrelated host cannot.
- **Egress allowlist.** The agent's own HTTP/browser client is restricted to the target host; any attempt to reach another host raises `SecurityError`. This is the primary defence against a prompt-injected response trying to redirect the agent.
- **Request budget.** A hard per-run cap (default 500), enforced at the tool-call boundary, not as a soft warning.
- **Untrusted target output.** Content read back from the target is treated strictly as data to evaluate, never as instructions to the agent.
- **Full logging.** Every request/response is logged to a per-run JSONL file for reproducibility and PoC generation.

8. Limitations (scope boundaries, stated honestly)

Framing these as principled boundaries — not silent underperformance — is deliberate (per `markdown/07_EVALUATION.md`):

1. **Business-logic / multi-step flaws are out of scope.** Race conditions, workflow-state abuse, and chained logic bugs lack a single checkable success condition and are not targeted. The system is strong precisely where deterministic verification is possible.
2. **Three classes are code-complete + unit-tested but not live-confirmed on Juice Shop:** blind SQLi (needs a DB with `SLEEP()`, e.g. DVWA/MySQL), OS-file path traversal (needs a file-read sink, e.g. DVWA file inclusion), and SSRF (needs a sandbox-internal callback listener). Their verifier logic is tested with mocked responses; a live true-positive needs the appropriate target/infra.
3. **ZAP active-scan comparison not run on the evaluation host** (RAM; Section 6.3). Passive results are reported with that caveat; the active runner is ready for a larger machine.
4. **The LLM-judged Verifier-Agent fallback was not needed.** Deterministic checks resolved every case, so the `llm_judged` tier appears only as a conservative "absence of proof is not proof" branch in the verifier logic; a standalone fallback agent was not built.
5. **Primarily single-target (Juice Shop).** DVWA/WebGoat are supported by the design but not exercised live here.
6. **State persistence is in-memory.** Durable persistence (`AsyncPostgresSaver`) is a deferred infrastructure upgrade, correctly sequenced after the core loop was proven.
7. **LLM variance** is measured here (stable at ±0.00 across 3 seeds on 4 classes) but could differ on larger/harder surfaces; the engine-mode baseline exists precisely to decouple the verifier's guarantee from the agent's behaviour.

9. Future Work

- Run the ZAP active-scan head-to-head on a higher-RAM host (one command; Section 11).
- Add DVWA + an SSRF callback listener to the sandbox to live-confirm blind SQLi, path traversal, and SSRF.
- A fair RAG ablation on a harder target where base recall < 1.0.
- An auth-bypass verifier so logical bypasses (e.g. `' OR 1=1--`) are *classified*, not merely "wrong payload" for error-based SQLi.
- Durable state (`AsyncPostgresSaver`) and LangSmith tracing for the reasoning trace.
- Continue the orchestration past the first finding to sweep the whole attack surface.

10. Defense preparation (likely viva questions)

Q. How does AutoRedTeam avoid the false positives that plague scanners? It never reports a *hypothesis*. A finding exists only after the verifier *executes* a PoC and a deterministic condition holds (structural data diff for IDOR, baseline-differenced error signature for SQLi, real DOM execution for XSS, reject-then-accept for broken auth). Measured deterministic false-positive rate: 0.00.

Q. Isn't this just the LLM deciding what's vulnerable? No. The LLM (Attacker) only *proposes* a typed hypothesis; a separate, LLM-free engine *judges* it by execution. Proposing and judging are different components with a schema boundary between them. The engine-mode results (no LLM at all) prove the verification stands on its own.

Q. How do you know the 1.00 recall wasn't a lucky run? Two ways: engine mode is deterministic and reproduces exactly; agent mode was run across 3 seeds and is stable at 1.00 ± 0.00 precision/recall. Only effort varied (1.08 ± 0.12 attempts).

Q. How does the sandbox prevent real-world harm? The target is on an `internal: true` Docker network with no egress; the agent's tools are host-allowlisted; there's a hard request cap; and target responses are treated as data, not instructions. All verified.

Q. Why is your overall recall only 0.57? Because that denominator includes three classes the system deliberately does not target on this target (no SQLite `SLEEP`, confined file access, no OOB listener). On the in-scope classes, recall is 1.00. The 0.57 is honesty, not weakness — it's reported as explicit false negatives.

Q. Why did ZAP look so different? Different philosophy: ZAP reports heuristic alerts (238, passive) for a human to triage; AutoRedTeam reports only execution-confirmed findings (4, each with a PoC). The comparison shown is passive-only due to a host-memory limit, and that caveat is stated.

Q. What's the weakest part? The agent's payload choice is stochastic (we mitigate with retries + multi-seed reporting), and the deterministic approach is, by design, blind to business-logic vulnerabilities.

11. Reproducibility appendix

```
# 1. Sandbox (isolated, egress-blocked Juice Shop)
docker compose -f docker-compose.sandbox.yml up -d          # target on 127.0.0.1:3000

# 2. Unit tests (no Docker / LLM / browser needed)
python -m pytest -q                                          # 57 passing

# 3. Per-phase live demos (LLM via Groq, OpenAI-compatible)
export AUTOREDTEAM_LLM_BASE_URL=https://api.groq.com/openai/v1
export AUTOREDTEAM_LLM_MODEL=openai/gpt-oss-120b
export GROQ_API_KEY=...                                      # a funded/eligible key
python scripts/run_recon_live.py            # Phase 2: 76 endpoints
python scripts/run_attacker_live.py         # Phase 3: agent-driven IDOR, end-to-end
python scripts/run_orchestration_live.py    # Phase 4: single-call verified finding
python scripts/run_phase5_live.py           # Phase 5: 4/6 classes true-positive
python scripts/run_rag_demo.py              # Phase 6: scoped retrieval + ablation

# 4. Benchmark (engine mode + 3 agent seeds + RAG ablation)
python -m evals.run_benchmark               # live precision/recall/FP/attempts

# 5. API + dashboard
uvicorn api.main:app --port 8077            # then set dashboard API URL to :8077
streamlit run dashboard/app.py

# 6. OWASP ZAP head-to-head (needs a higher-RAM host for the ACTIVE scan)
docker run -d --name redteam-zap --network redteamingagent_redteam-edge \
  -p 127.0.0.1:8090:8090 zaproxy/zap-stable zap.sh -daemon -host 0.0.0.0 -port 8090 \
  -config api.disablekey=true -config api.addrs.addr.name=.* -config api.addrs.addr.regex=true
docker network connect redteamingagent_redteam-isolated redteam-zap
ZAP_API_URL=http://127.0.0.1:8090 python scripts/run_zap_only.py          # active (default)
ZAP_PASSIVE_ONLY=1 ZAP_API_URL=http://127.0.0.1:8090 python scripts/run_zap_only.py  # passive
```